

Appendix to “Augmenting Multi-Technique Static Analysis with Large Language Models: A Neuro-Symbolic Approach to Smart Contract Vulnerability Detection”

JUNXIANG WANG, Tongji University, China

FU SONG*, Key Laboratory of System Software (Chinese Academy of Sciences), China, Institute of Software at Chinese Academy of Sciences, China, and University of Chinese Academy of Sciences, China

MIAOMIAO ZHANG†, Tongji University, China

BOWEN DU, Tongji University, China

RONGCAN PEI, Tongji University, China

Smart contracts facilitate and enforce agreements between untrusted parties without trusted intermediaries, but vulnerabilities within contracts can cause severe damage once exploited. Various analysis techniques have been proposed for vulnerability detection, but they are typically limited to specific vulnerability types. While recent frameworks employ aggregation to broaden detection capacity, they often accumulate false positives due to loose integration that fails to resolve underlying conflicts. Meanwhile, the semantic reasoning capacity of large language models (LLMs) has shown promise in detecting vulnerabilities, despite inherent reasoning bottlenecks and hallucinations. Recognizing these challenges, in this work, we propose a novel neuro-symbolic approach, named Ensemble LLM-assisted Static Analysis (ELSA). ELSA comprises two key modules, namely, LLM-assisted static analysis and analyzer ensemble (the ensemble of multiple LLM-assisted static analyzers), each incorporating two distinct strategies. The LLM-assisted static analysis augments individual analysis techniques with constraint-guided neural semantic reasoning, while the analyzer ensemble resolves conflicting outputs to distill a robust consensus. We evaluate ELSA on a comprehensive benchmark, including three open-source datasets and additional self-constructed Zero-Knowledge Proof-based smart contracts whose complexity poses unique challenges to static analysis. Experimental results demonstrate that our approach achieves an overall improvement of at least 17% over baselines and advanced mainstream approaches, effectively bridging semantic gaps and synergizing the complementary advantages of different analyzers. Furthermore, an ablation study and fine-grained analysis are conducted to investigate the key factors contributing to overall performance gains.

CCS Concepts: • **Software and its engineering** → **Software defect analysis**; • **Security and privacy** → **Software and application security**; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: Smart contracts, Zero-knowledge proofs, Large language models, Static analysis, Vulnerability detection

*Also with Nanjing Institute of Software Technology.

†Corresponding author.

Authors' Contact Information: Junxiang Wang, tongji_junxiangwang@tongji.edu.cn, Tongji University, Shanghai, China; Fu Song, songfu@ios.ac.cn, Key Laboratory of System Software (Chinese Academy of Sciences), Beijing, China and Institute of Software at Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China; Miaomiao Zhang, miaomiao@tongji.edu.cn, Tongji University, Shanghai, China; Bowen Du, bowendu@tongji.edu.cn, Tongji University, Shanghai, China; Rongcan Pei, prc@tongji.edu.cn, Tongji University, Shanghai, China.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/10-ARTISSTA131

<https://doi.org/10.1145/3832222>

ACM Reference Format:

Junxiang Wang, Fu Song, Miaomiao Zhang, Bowen Du, and Rongcan Pei. 2026. Appendix to “Augmenting Multi-Technique Static Analysis with Large Language Models: A Neuro-Symbolic Approach to Smart Contract Vulnerability Detection”. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA131 (October 2026), 15 pages. <https://doi.org/10.1145/3832222>

.1 Detailed Code Examples for Prompt Engineering

Figure 1 presents detailed code examples illustrating the prompt engineering process described in our approach, including the specific prompts used in each strategy. The upper portion of the figure corresponds to the One-Shot LLM-assisted analysis strategy, whereas the lower portion depicts the Chain-of-Thought reasoning-enhanced LLM analysis strategy. To facilitate consistent and accurate evaluation, the prompts are designed to constrain the detection scope to specific vulnerability types and adopt a standardized output format, namely a list of high-risk expressed in terms of SWC codes [14]. We also provide representative LLM output examples to ensure stable and interpretable outputs. It should be noted that the Chain-of-Thought reasoning-enhanced LLM analysis strategy incorporates detailed prompt designs across its three analysis stages. In particular, during the second stage, we introduce specific prompts for the Analysis Process to further guide the output and improve the quality of the results. Additionally, as shown in Figure 2, instead of direct detection, Zero-Knowledge Proofs (ZKP)-specific LLM analysis is optimized for constraint-to-logic translation, generating semantic hints that map implicit mathematical constraints to high-level security invariants, thus providing precise guidance for subsequent detection.

.2 Construction of the Benchmark Dataset

We construct a benchmark dataset to evaluate the feasibility and generalizability of ELSA. The dataset includes both ZKP-based smart contracts and three public benchmark datasets named SmartBugs Curated (SmartBugs) [3], SolidiFI Benchmark (SolidiFI) [5], and DAppSCAN [17]. The following provides a detailed introduction to their composition.

Dataset for ZKP-based Smart Contracts. The integration of Zero-Knowledge Proof (ZKP) frameworks into smart contracts significantly increases their structural complexity and reduces transparency, thereby making vulnerability detection much more challenging [13, 15]. However, due to the novelty of ZKP applications, existing open-source datasets for smart contracts lack sufficient coverage of ZKP-related content, and only four ZKP-based smart contracts have been found on Ethereum, which makes it difficult to evaluate the feasibility of our proposed strategies across a wider range of smart contracts. To address this gap, we introduce a dedicated vulnerability dataset targeting ZKP-based smart contracts. The dataset is constructed through vulnerability injection, whose original data includes 23 single-chain and 12 cross-chain ZKP-based smart contracts. These contracts are sourced from open-source projects on Ethereum and GitHub related to ZKP-based smart contracts. To ensure these contracts are compatible with downstream analysis techniques, we manually combine the core project files with the corresponding third-party library code, resulting in fully executable smart contract files.

Through our research, we identify the following types of vulnerabilities as particularly critical in ZKP-based smart contracts [4, 8, 15, 18]: Integer Overflow and Underflow, Unprotected Ether Withdrawal, Reentrancy, Assert Violation, Missing Protection Against Signature Replay Attacks, Write to Arbitrary Storage Location, and Denial of Service (DoS) with Block Gas Limit. It is important to note that although these vulnerabilities can also exist in smart contracts without ZKP, their impact is significantly more severe if they occur in ZKP-based smart contracts. Therefore, we independently design templates for each of the seven identified vulnerability types and inject them into the original ZKP-based smart contract data, using appropriate security vulnerability

One-Shot LLM-Assisted Analysis Strategy Prompting [Specification Prompt]**TASK:**

Analyze the following Solidity code and mythril output to identify vulnerability types and return the highest risk vulnerability type. Detection scope includes:

'SWC-101': Integer Overflow and Underflow, 'SWC-105': Unprotected Ether Withdrawal, 'SWC-107': Reentrancy, 'SWC-110': Assert Violation, 'SWC-121': Missing Protection Against Signature Replay Attacks, 'SWC-124': Write to Arbitrary Storage Location, 'SWC-128': DoS with Block Gas Limit Gas.

- Contract Structure Hint: {structure_hint} - Solidity File Content: {solidity_response}
- Mythril Output: {mythril_output_str}

Return format: [SWC Code]: Line Number: [Line Number]. Example: SWC-101: Line 52 \n SWC-107: Not Found

If no vulnerabilities are found, the format is: SWC-000: No vulnerabilities found.

Please identify any potential vulnerabilities and return the corresponding SWC code list.

Chain-of-Thought Reasoning-Enhanced LLM Analysis Strategy Prompting [Specification Prompt]**TASK:****STEP 1: Heuristic semantic prior generation.**

Based on the following Solidity file content, summarize and validate accurate and effective cues from the three provided hints to guide GPT analysis. If a vulnerability does not exist in the Solidity file, please remove it. Return the high-risk function name and the associated issue.

- Contract Structure Hint: {structure_hint} - Possible Risk Hint: {property-based_semantic_identification}
- Analysis Hint: {zkp_model_hint} - Solidity File Content: {solidity_response}

STEP 2: Integration with technique analysis.

Analyze vulnerabilities in the following Solidity file, using risk function clues from the inspiration hints. Return the highest risk vulnerability type. Summarize and confirm existing SWC code vulnerabilities. Do not provide fix suggestions or detection ranges. Ensure the vulnerabilities exist.

Detection range:

'SWC-101': Integer Overflow and Underflow, 'SWC-105': Unprotected Ether Withdrawal, 'SWC-107': Reentrancy, 'SWC-110': Assert Violation, 'SWC-121': Missing Protection Against Signature Replay Attacks, 'SWC-124': Write to Arbitrary Storage Location, 'SWC-128': DoS with Block Gas Limit Gas.

Analysis Process:

Check each function individually to identify potential vulnerabilities, focusing on features related to access control, arithmetic operations, external calls, etc. Ensure that the returned SWC codes are accurate, and specify the exact function or code segment where the vulnerability is located.

- Contract Structure Hint: {structure_hint} - Solidity File Content: {solidity_response}
- Mythril Output: {slither_output} - Inspiration Hint: {inspiration_prior_response}

Return format: [SWC Code]: Line Number: [Line Number]. Example: SWC-101: Line 52 \n SWC-107: Not Found

If no vulnerabilities are found, the format is: SWC-000: No vulnerabilities found.

STEP 3: Semantic execution verification.

Based on the logic of symbolic execution techniques, and combining the following Solidity code and discovered vulnerabilities, return the highest-risk vulnerability type. Simulate execution paths of the smart contract under different inputs, inspect potential execution risks, and verify the validity of discovered vulnerabilities. Remove vulnerabilities that do not actually exist. Discovered vulnerabilities: {'.'.join(vulnerabilities_found)}

Detection range:

'SWC-101': Integer Overflow and Underflow, 'SWC-105': Unprotected Ether Withdrawal, 'SWC-107': Reentrancy, 'SWC-110': Assert Violation, 'SWC-121': Missing Protection Against Signature Replay Attacks, 'SWC-124': Write to Arbitrary Storage Location, 'SWC-128': DoS with Block Gas Limit Gas.

- Contract Structure Hint: {structure_hint} - Solidity File Content: {solidity_response}

Return format: [SWC Code]: Line Number: [Line Number]. Example: SWC-101: Line 52 \n SWC-107: Not Found

If no vulnerabilities are found, the format is: SWC-000: No vulnerabilities found.

Please identify any potential vulnerabilities and return the corresponding SWC code list.

Fig. 1. Detailed prompt templates corresponding to the One-Shot LLM-assisted analysis strategy (top) and the Chain-of-Thought reasoning-enhanced LLM analysis (bottom), both extensible to new vulnerability categories.

templates from a predefined pool. Each vulnerability template is injected as a complete function, as shown in Figure 3, with the injection positions randomized to promote dataset diversity and avoid duplication. These vulnerability templates were synthesized from the official SWC-registry [14]

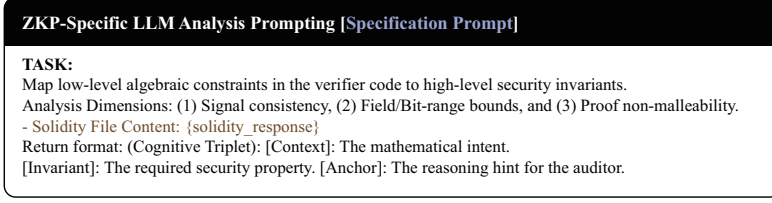


Fig. 2. Detailed code examples for the prompt engineering corresponding to ZKP-specific LLM analysis.

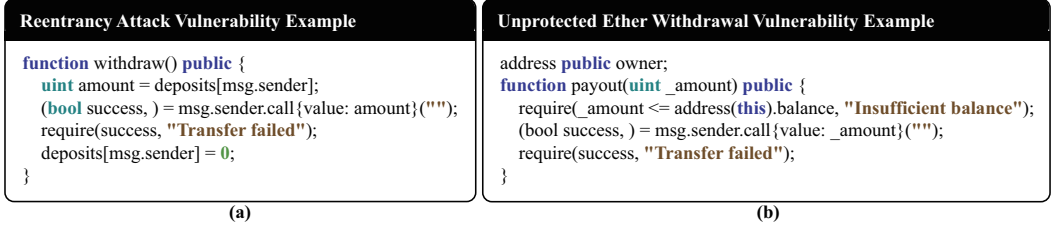


Fig. 3. Examples of vulnerability templates in the vulnerability pool. (a) shows an example of a reentrancy attack vulnerability. (b) shows an example of unprotected ether withdrawal vulnerability.

and the patterns summarized in recent ZKP security literature [4, 8, 15, 18]. To ensure data quality, the files containing the injected vulnerabilities are required to compile successfully. Additionally, at the beginning of each file with injected vulnerabilities, we annotate the type and location of the vulnerabilities in the form of comments.

Finally, the specialized dataset we construct, focused on vulnerabilities within ZKP-based smart contracts, contains 2,310 Solidity files, as shown in Table 1. Of these, 2,205 files are generated using the vulnerability injection method described earlier (315 files per vulnerability type, 9 variants per original contract). The remaining 105 files are GPT-generated to diversify the dataset (15 per vulnerability type). To facilitate our analysis, we performed sampling and narrowed the testing focus to 224 randomly selected Solidity files from the benchmark dataset. The test dataset is categorized by vulnerability type and employs sampling with a 95% confidence interval and a 6.56% margin of error.

Table 1. The composition of the dataset for ZKP-based smart contracts.

Vulnerability type	Cross chain	Single chain	GPT	Total
Integer Overflow and Underflow	12*9=108	23*9=207	15	330
Unprotected Ether Withdrawal	12*9=108	23*9=207	15	330
Reentrancy	12*9=108	23*9=207	15	330
Assert Violation	12*9=108	23*9=207	15	330
Missing Protection Against Signature Replay Attacks	12*9=108	23*9=207	15	330
Write to Arbitrary Storage Location	12*9=108	23*9=207	15	330
DoS with Block Gas Limit	12*9=108	23*9=207	15	330
Total	756	1449	105	2310

SmartBugs Curated Dataset. The SmartBugs Curated dataset [3] is designed for detecting vulnerabilities in smart contracts and is organized according to the DASP taxonomy [7]. It contains 143 vulnerable Solidity smart contracts, specifically curated for automated reasoning and testing of smart contracts. The dataset covers 10 types of vulnerabilities, including Reentrancy, Access Control, Arithmetic, Unchecked Low Level Calls, and others. All contracts in the dataset are sourced from public websites or obtained via Etherscan.

Table 2. Evaluation of LLM-assisted static analysis strategies on the clean contracts.

Strategy	Mythril					Slither					SmartCheck				
	TN	FP	TNR	AFPC	FPR	TN	FP	TNR	AFPC	FPR	TN	FP	TNR	AFPC	FPR
Baseline	83.3%	16.7%	83.3%	1.00	16.7%	5.6%	94.4%	5.6%	2.41	94.4%	0.0%	100.0%	0.0%	1.89	100.0%
Slight	83.3%	16.7%	83.3%	1.00	16.7%	5.6%	94.4%	5.6%	2.41	94.4%	0.0%	100.0%	0.0%	1.89	100.0%
One-Shot	77.8%	22.2%	77.8%	1.00	22.2%	38.9%	61.1%	38.9%	2.09	61.1%	61.1%	38.9%	61.1%	1.86	38.9%
CoT	83.3%	16.7%	83.3%	1.00	16.7%	38.9%	61.1%	38.9%	1.00	61.1%	33.3%	66.7%	33.3%	1.00	66.7%

Table 3. Evaluation of LLM-assisted static analysis strategies combined with analyzer ensemble strategies on clean contracts.

Strategy	WI					OS					VDC
	TN	FP	TNR	AFPC	FPR	TN	FP	TNR	AFPC	FPR	
Baseline	5.6%	94.4%	5.6%	1.00	94.4%	83.3%	16.7%	83.3%	1.00	16.7%	100.0%
Slight	33.3%	66.7%	33.3%	1.00	66.7%	83.3%	16.7%	83.3%	1.00	16.7%	100.0%
One-Shot	72.2%	27.8%	72.2%	1.00	27.8%	77.8%	22.2%	77.8%	1.00	22.2%	72.2%
CoT	66.7%	33.3%	66.7%	1.00	33.3%	83.3%	16.7%	83.3%	1.00	16.7%	83.3%

SolidiFI Benchmark Dataset. The SolidiFI Benchmark repository [5] consists of flawed smart contracts created by injecting vulnerabilities into original contracts. The vulnerabilities in the dataset are primarily related to seven different types of defects, including timestamp dependency, unhandled exceptions, and unchecked send. The vulnerability pool comprises a total of 9,369 defects. For consistency in evaluation, TOD vulnerabilities are excluded from the experiments, as they are difficult to detect with most baseline techniques, offering limited relevance to research objectives.

DAppSCAN Dataset. The DAppSCAN Dataset [17], developed by InPlusLab, focuses on real-world DApp projects that have undergone formal security audits, addressing limitations of synthetic datasets. It covers 37 vulnerability types across domains such as DeFi, NFT, and DEX, with hundreds of audited Solidity contracts and 608 audit reports. Compared to other sub-datasets, DAppSCAN better reflects the complexity and diversity of production-grade smart contracts, making it an essential benchmark for evaluating vulnerability detection methods.

3 Evaluation on clean contracts

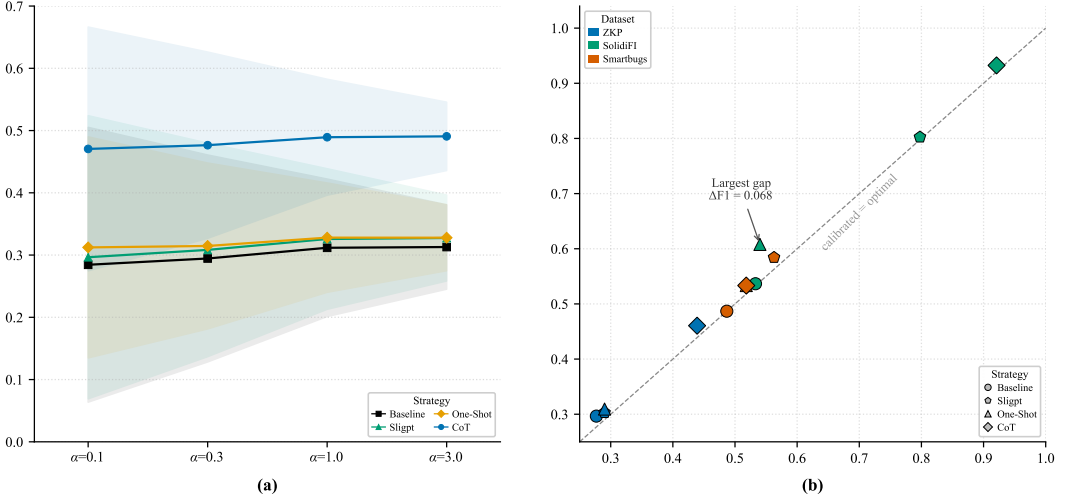
To evaluate false alarm suppression, we used a clean benchmark of 18 non-vulnerable contracts, selected based on the manual verification results in Section 4.1 of the main text. As shown in Table 2, baseline techniques like Slither and SmartCheck generate excessive false positives. However, both proposed LLM-assisted static analysis strategies effectively mitigate this issue. Overall, One-Shot achieves the best performance in True Negative Rate (TNR), followed closely by CoT. Meanwhile, CoT distinguishes itself by reducing AFPC to 1.00, thereby preventing multi-alarm noise. In contrast, Slight's over-reliance on the LLM's judgment results in virtually no improvement over the baseline techniques. This trend extends to the analyzer ensemble module (Table 3). One-Shot similarly yields the best performance under the weighted integration (WI) strategy, while its slight TNR dip under the optimal selection (OS) strategy reflects a minor regression on the Mythril baseline. Notably, in this clean benchmark, the VDC denotes union-based false-alarm accumulation rather than true vulnerability detection coverage. Under this metric, our strategies demonstrate a clear reduction in noise propagation compared to the baseline techniques. These findings validate our approach's efficacy in suppressing false alarms on true negative samples.

4 Comparison with Alternative Analyzer Ensemble Strategies

As demonstrated in the main text, the proposed analyzer ensemble strategies (i.e., weighted integration and optimal selection) can be combined with LLM-assisted static analysis strategies to yield a substantial synergistic effect compared to applying them independently.

Table 4. Comparison of alternative analyzer ensemble strategies on the sub-dataset for ZKP-based smart contracts.

Strategy	Game-Theoretic					Randomization				
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score
Baseline	16.07%	0.00%	83.93%	0.00	0.28	13.94%	0.00%	86.06%	0.00	0.24
Sligpt	16.96%	0.00%	83.04%	0.00	0.29	15.25%	0.00%	84.75%	0.00	0.26
One-Shot	9.38%	0.00%	90.62%	0.00	0.17	14.16%	0.00%	85.84%	0.00	0.25
CoT	26.79%	0.00%	73.21%	0.00	0.42	28.28%	0.00%	71.72%	0.00	0.44

Fig. 4. Sensitivity analysis of the WI strategy from two perspectives. (a) F1-score stability under skewed vulnerability distributions (concentration $\alpha \in \{0.1, 0.3, 1.0, 3.0\}$). (b) Cross-dataset generalizability (gap between calibrated weights and empirical optima).

To further validate the competitiveness of the weighted integration (WI) strategy relative to other distribution-agnostic ensemble paradigms, we benchmark it against the Game-Theoretic approach (using Nash-optimal worst-case weights [9, 16] derived via 5-fold cross-validation to prevent data leakage) and Randomization (using random Dirichlet weights [12] averaged over 5,000 runs for convergence) on the sub-dataset for ZKP-based smart contracts. As shown in Tables 4 and 9 (a), WI achieves highly competitive performance and superior stability. By contrast, the Game-Theoretic approach suffers from instability due to heavy data reliance, which manifests as a drastic degradation in One-Shot’s performance. Moreover, the Randomization method shows no clear advantage. Its combination with CoT merely matches that of WI, and its other combinations underperform their WI counterparts. These findings validate the efficacy and stability of WI in synergizing multi-technique outputs.

5 Sensitivity Analysis of Weighted Integration

In the main text, the weights for Weighted Integration (WI) are calibrated using a balanced dataset. However, real-world data scenarios are rarely uniform. To investigate the strategy’s generalizability to unknown environments, we conduct sensitivity analyses from two perspectives: skewed vulnerability distributions and cross-dataset validation.

We sample the proportions of the seven vulnerability categories from a Dirichlet distribution with diverse concentration parameters $\alpha \in \{0.1, 0.3, 1.0, 3.0\}$. As shown in Figure 4 (a), the mean F1-score remains stable over 3,000 draws, demonstrating WI’s robustness against varying degrees

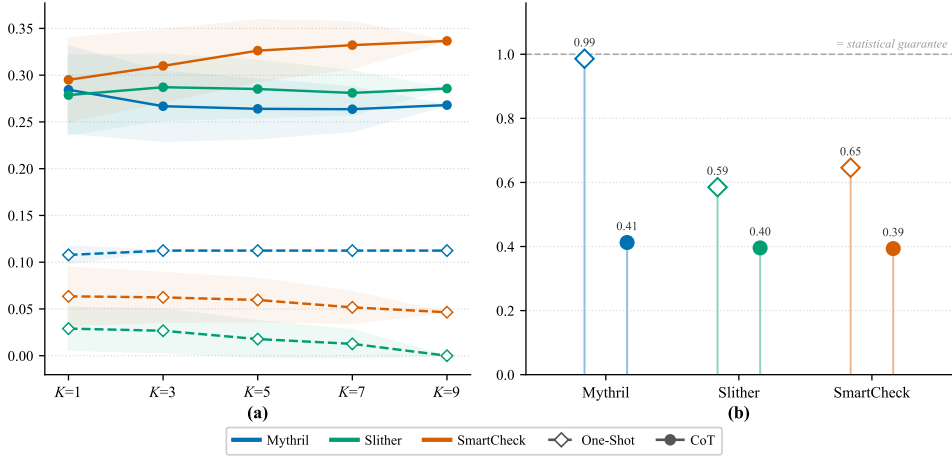


Fig. 5. Impact of trial count K in the probabilistic verification layer across different pairings of static analyzers and LLM-assisted analysis strategies. (a) Mean F1-score stability and variance convergence across $K \in \{1, 3, 5, 7, 9\}$ (shaded bands represent ± 1 standard deviation). (b) Inter-trial agreement (mean pairwise Jaccard similarity, where 1.0 denotes perfect deterministic agreement).

of vulnerability skewness. We further evaluate the transferability of the calibrated weights across sub-datasets. For each sub-dataset, we compare its performance against the grid-searched empirical upper bound, excluding DAppSCAN due to its sparse vulnerability signals. Figure 4 (b) shows only small gaps between these weights and the empirical optima across sub-datasets, suggesting that WI maintains near-optimal performance under imbalanced distributions. Overall, these results show that WI is robust to distributional shifts.

.6 Impact of K in the Probabilistic Verification Layer

The probabilistic verification layer runs K independent trials $\mathcal{R} = (r_1, r_2, \dots, r_K)$ to achieve consensus-based stabilization. However, the impact of K on performance and stability still requires quantitative evaluation. Therefore, we evaluated different K values across various pairings of static analyzers and LLM-assisted analysis strategies using 84 contracts sampled from the ZKP-based sub-dataset, with near-uniform category coverage ($\sigma = 2.27$ across category sizes).

As shown in Figure 5 (a), increasing K improves SmartCheck under CoT, but has limited effects on other pairings. However, the narrowing shaded bands demonstrate that increasing K effectively reduces performance variance across all pairings. This suggests that although the benefit of the probabilistic verification layer depends on the specific characteristics of the evaluated pairings, the layer can effectively suppress the inherent stochasticity of LLMs. Figure 5 (b) reports the mean pairwise Jaccard similarity across trials. The observed agreement indicates a certain degree of overlap among the outputs, which enables the probabilistic verification layer. Considering both performance and cost, we set $K=3$ for this layer, as further increases yield limited incremental gains and marginal improvements in variance reduction (Figure 5 (a)). Additionally, the consensus-based verification iterates for at most five rounds, dynamically terminating early once the majority agreement is reached.

.7 Comparative Test for the Most Suitable LLM

In the main text, we briefly elaborated on the reasons for selecting GPT-3.5-turbo as the most suitable LLM. In this section, we provide a more detailed explanation.

Given the high costs associated with models like GPT-4o and the increased instability observed in our experiments (shown in Table 5), unless it is heavily constrained, we focused our comparison on GPT-3.5-turbo and the latest DeepSeek-V3, as both offer a strong balance of performance and cost-effectiveness. To determine the most suitable LLM for our proposed strategies, we compare their performance when assisted by different LLMs. This evaluation was conducted on the sub-dataset for ZKP-based smart contracts from two perspectives: accuracy and token consumption, with the results compared against those reported in the main text.

Table 5. Comparison of GPT-3.5-turbo and GPT-4o for vulnerability detection in smart contracts across four sub-datasets.

Strategy	ZKP					Smartbugs					SolidfI					DAppSCAN				
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score
GPT-3.5-turbo	8.48%	0.45%	91.07%	4.00	0.16	11.19%	15.38%	73.43%	1.14	0.20	11.67%	15.33%	73.00%	3.91	0.21	1.69%	5.08%	93.23%	3.00	0.03
GPT-4o	4.46%	3.57%	91.96%	1.00	0.09	8.39%	23.08%	68.53%	3.18	0.16	5.00%	1.67%	93.33%	1.00	0.10	0.00%	33.90%	66.10%	4.55	0.00

Accuracy. When DeepSeek-V3 is selected as the LLM in our approach, both LLM-assisted static analysis strategies still outperform the baseline techniques, as shown in Figure 6 (a). The analysis characteristics of both strategies using DeepSeek-V3 are similar to those observed with the GPT-3.5-turbo. However, DeepSeek-V3 introduces more false positives under the Chain-of-Thought reasoning-enhanced LLM analysis strategy, despite detecting more vulnerabilities. We attribute this phenomenon to over-interpretation. Ideally, LLMs should refine static-analysis results through neural semantic reasoning, whereas reasoning-oriented models often go beyond the provided evidence, inferring plausible but insufficiently supported vulnerabilities from benign warnings or partial traces. Additionally, the analyzer ensemble strategies remain equally effective when combined with LLM-assisted static analysis strategies, as shown in Figure 6 (b). However, their overall performance was suboptimal, primarily due to the inadequate performance of LLM-assisted static analysis strategies.

Table 6. Evaluation on the sub-dataset for ZKP-based smart contracts with DeepSeek-V3.

(a) LLM-assisted static analysis strategies.

Strategy	Mythril					Slither					SmartCheck				
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score
Baseline	4.02%	7.14%	88.84%	1.12	0.08	8.04%	2.23%	89.73%	1.00	0.15	7.59%	0.00%	92.41%	0.00	0.14
One-Shot	12.95%	8.93%	78.12%	1.00	0.23	29.46%	8.04%	62.50%	1.06	0.46	26.79%	6.25%	66.96%	1.14	0.42
CoT	3.58%	30.80%	65.62%	1.62	0.07	2.68%	38.84%	58.48%	2.33	0.05	2.68%	38.84%	58.48%	2.15	0.05

(b) LLM-assisted static analysis strategies combined with analyzer ensemble strategies.

Strategy	WI					OS					VDC
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	
Baseline	16.07%	0.00%	83.93%	0.00	0.28	16.52%	3.12%	80.36%	1.14	0.28	25.89%
One-Shot	33.93%	0.00%	66.07%	0.00	0.51	37.95%	10.71%	51.34%	1.00	0.55	52.68%
CoT	23.21%	0.00%	76.79%	0.00	0.38	3.57%	39.29%	57.14%	2.27	0.07	47.32%

Token consumption. We also measured the token consumption of the strategies under both LLMs. For the One-Shot LLM-assisted analysis strategy, the average token consumption per file is approximately 25,687.48 tokens if GPT-3.5-turbo is selected as the model. In comparison, the average token consumption per file is about 13,272.88 tokens if DeepSeek-V3 is used. For the Chain-of-Thought reasoning-enhanced LLM analysis strategy, the average token consumption per file is approximately 66,480.02 tokens with GPT-3.5-turbo, while it is around 33,129.77 tokens if DeepSeek-V3 is selected. According to the corresponding price lists [1, 2], we found that the cost of using DeepSeek-V3 is lower. It makes the average consumption per file more economical. However, for analyzing a single file, the token consumption of both models falls within an acceptable range.

Given these findings, we recommend selecting the model with higher accuracy for the experimental analysis. Therefore, based on its superior performance and lower cost, GPT-3.5-turbo is selected as the most suitable LLM for our approach, in contrast to DeepSeek-V3.

.8 Generalization of Strategies in the LLM-Assisted Static Analysis Module and Analyzer Ensemble Module

We employed additional techniques to verify the generalization of the strategies we proposed. The evaluation was conducted on the SmartBugs Curated (SmartBugs) [3], SolidiFI Benchmark (SolidiFI) [5], and DAppSCAN sub-datasets, as the sub-dataset for ZKP-based smart contracts uses a relatively new compilation version ($\geq 0.8.0$), which is not compatible with the additional techniques.

As shown in Table 7, both strategies in the LLM-assisted static analysis module, along with the strategy from Sligpt, outperformed the baseline techniques. On the SmartBugs Curated sub-dataset, the One-Shot LLM-assisted analysis strategy achieved the best performance, with Manticore performing the best (analyzed in combination with the results from RQ1). Notably, this contradicts the conclusions in the main text: except for Mythril, the One-Shot LLM-assisted analysis strategy outperformed Sligpt's strategy, achieving the highest performance on the SmartBugs Curated sub-dataset. This suggests that the One-Shot LLM-assisted analysis strategy is the most effective overall when baseline techniques fall short. Meanwhile, on the SolidiFI Benchmark sub-dataset, the Chain-of-Thought reasoning-enhanced LLM analysis strategy maintained the best performance, with Oyente performing the best (analyzed in combination with the results from RQ1). These findings demonstrate the generalization of our proposed LLM-assisted static analysis strategies.

Table 7. Generalization test of the strategies in the LLM-assisted static analysis module across two publicly available sub-datasets.

(a) Generalization of LLM-assisted static analysis strategies on the SmartBugs Curated sub-dataset.

Strategy	Honeybadger					Manticore					Osiris					Oyente				
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score
Baseline	0.70%	1.40%	97.90%	2.00	0.01	14.10%	3.50%	82.40%	1.00	0.26	7.69%	14.69%	77.62%	1.14	0.14	12.58%	14.69%	72.73%	1.14	0.22
Sligpt	35.66%	2.80%	61.54%	1.00	0.53	37.76%	1.40%	60.84%	1.00	0.55	37.76%	0.00%	62.24%	0.00	0.55	40.56%	0.70%	58.74%	1.00	0.58
One-Shot	38.46%	2.10%	59.44%	1.00	0.56	47.55%	0.00%	52.45%	0.00	0.64	36.36%	0.00%	63.64%	0.00	0.53	44.06%	0.00%	55.94%	0.00	0.61
CoT	30.07%	10.49%	59.44%	1.07	0.46	33.56%	4.20%	62.24%	1.00	0.50	33.56%	5.59%	60.85%	1.00	0.50	32.16%	3.50%	64.34%	1.00	0.49

(b) Generalization of LLM-assisted static analysis strategies on the SolidiFI Benchmark sub-dataset.

Strategy	Manticore					Securify					Oyente				
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score
Baseline	30.00%	3.33%	66.67%	1.00	0.47	24.67%	2.67%	72.66%	1.00	0.40	12.00%	1.00%	87.00%	1.00	0.22
Sligpt	59.33%	0.00%	40.67%	0.00	0.74	70.67%	0.00%	29.33%	0.00	0.83	62.00%	0.00%	38.00%	0.00	0.77
One-Shot	68.00%	0.00%	32.00%	0.00	0.81	36.00%	0.00%	64.00%	0.00	0.53	98.00%	0.00%	2.00%	0.00	0.99
CoT	80.67%	0.00%	19.33%	0.00	0.89	79.33%	0.00%	20.67%	0.00	0.88	99.00%	0.00%	1.00%	0.00	0.99

The ensemble analysis strategies also remain effective when combined with LLM-assisted static analysis strategies, as shown in Table 8. It can be seen that performance improved overall with the inclusion of additional baseline techniques, as they helped fill the gaps in detecting vulnerabilities left by existing baseline techniques, resulting in a more comprehensive analysis. This further confirms the generalization of our analyzer ensemble strategies.

However, we observe that although the analyzer ensemble strategies with more baseline techniques generally outperformed those with fewer on the SolidiFI Benchmark sub-dataset, they still face the issue of diluting the contribution of the best-performing technique, leading to a slight decrease in overall performance compared to the best-performing technique. Another reason for this decrease is the change in overall evaluation scope resulting from combining techniques with distinct detection scopes for vulnerabilities. As noted in the paper accompanying the SolidiFI

Table 8. Generalization test of LLM-assisted static analysis strategies combined with analyzer ensemble strategies across two publicly available sub-datasets.

(a) Generalization of LLM-assisted static analysis strategies combined with analyzer ensemble strategies on the SmartBugs Curated sub-dataset.

Strategy	WI					OS					VDC
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	
Sligpt	37.06%	0.00%	62.94%	0.00	0.54	47.55%	0.00%	52.45%	0.00	0.64	60.14%
One-Shot	47.55%	0.00%	52.45%	0.00	0.64	58.74%	0.00%	41.26%	0.00	0.74	70.63%
CoT	35.66%	0.00%	64.34%	0.00	0.53	34.97%	8.39%	56.64%	1.08	0.52	51.75%

(b) Generalization of LLM-assisted static analysis strategies combined with analyzer ensemble strategies on the SolidiFI Benchmark sub-dataset.

Strategy	WI					OS					VDC
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	
Sligpt	68.33%	0.00%	31.67%	0.00	0.81	72.67%	0.00%	27.33%	0.00	0.84	80.67%
One-Shot	60.00%	0.00%	40.00%	0.00	0.75	67.67%	0.00%	32.33%	0.00	0.81	74.67%
CoT	85.67%	0.00%	14.33%	0.00	0.92	88.33%	0.00%	11.67%	0.00	0.94	92.67%

Benchmark sub-dataset [5], these scopes differ across techniques, as each technique is designed to detect different types of vulnerabilities. Consequently, the performance of analyzer ensemble strategies is assessed over the union of their detection scopes, rather than within each technique's individual scope. This broader evaluation may slightly lower reported performance metrics, even though the actual effectiveness of vulnerability detection is enhanced.

In general, although there are some variations in the results, the strategies in both the LLM-assisted static analysis module and the analyzer ensemble module still generalize well and improve performance with the inclusion of more techniques. Therefore, we also recommend incorporating more baseline techniques to improve the performance of our approach further.

9 Validation of RQ3 Effectiveness across Sub-Datasets

This section provides additional data for the evaluation of RQ3, which is presented in the main text. Table 9 presents a detailed evaluation of RQ3 across four sub-datasets. From the results, we can infer that the conclusions are consistent with those of RQ3 in the main text. On all four sub-datasets, combining the strategies from the LLM-assisted static analysis module and the analyzer ensemble module can achieve better results than using them independently. Notably, the weighted integration strategy effectively reduces false positives, while the optimal selection strategy better leverages the strengths of each technique, leading to distinct performance improvements. Additionally, the results suggest that Sligpt's LLM-assisted static analysis strategy relies more heavily on GPT, as indicated by the relatively low VDC value. It diminishes the individual strengths of each technique, which makes the multi-analysis integration approach less effective. In contrast, the two strategies in our LLM-assisted static analysis module more effectively leverage the advantages of each technique.

10 Detailed Precision and Recall Statistics

Due to space constraints, the metrics reported thus far are limited to True Positive (TP), False Positive (FP), False Negative (FN), Average False Positive Count (AFPC), and F1-score. We assess vulnerability detection and categorization at the contract level using these metrics, where the sum of TP and FP reflects overall contract-level recall, while AFPC serves as an inverse proxy for precision. However, they do not directly present explicit precision and recall. These two metrics can offer an alternative representation of the data for comparative analysis. Specifically, precision quantifies the proportion of reported vulnerabilities that are true positives, whereas recall measures

Table 9. Supplementary data for validating the effectiveness of RQ3 across four sub-datasets.

(a) Evaluation of LLM-assisted static analysis strategies combined with analyzer ensemble strategies on the sub-dataset for ZKP-based smart contracts.

Strategy	WI					OS					VDC
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	
Sligpt	16.96%	0.00%	83.04%	0.00	0.29	20.09%	1.34%	78.57%	1.00	0.33	25.89%
One-shot	16.96%	0.00%	83.04%	0.00	0.29	17.41%	0.89%	81.70%	1.00	0.30	28.57%
CoT	28.12%	0.00%	71.88%	0.00	0.44	33.04%	0.00%	66.96%	0.00	0.50	41.96%

(b) Evaluation of LLM-assisted static analysis strategies combined with analyzer ensemble strategies on the SmartBugs Curated sub-dataset.

Strategy	WI					OS					VDC
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	
Sligpt	39.16%	0.00%	60.84%	0.00	0.56	41.26%	0.70%	58.04%	1.00	0.58	48.25%
One-Shot	34.97%	0.00%	65.03%	0.00	0.52	46.15%	1.40%	52.45%	1.50	0.63	52.45%
CoT	34.97%	0.00%	65.03%	0.00	0.52	34.27%	7.69%	58.04%	1.00	0.51	44.76%

(c) Evaluation of LLM-assisted static analysis strategies combined with analyzer ensemble strategies on the SolidiFI Benchmark sub-dataset.

Strategy	WI					OS					VDC
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	
Sligpt	66.33%	0.00%	33.67%	0.00	0.80	69.33%	0.00%	30.67%	0.00	0.82	74.33%
One-Shot	37.00%	0.00%	63.00%	0.00	0.54	47.33%	0.00%	52.67%	0.00	0.64	55.33%
CoT	85.33%	0.00%	14.67%	0.00	0.92	88.00%	0.00%	12.00%	0.00	0.94	90.67%

(d) Evaluation of LLM-assisted static analysis strategies combined with analyzer ensemble strategies on the DAppSCAN sub-dataset.

Strategy	WI					OS					VDC
	TP	FP	FN	AFPC	F1-score	TP	FP	FN	AFPC	F1-score	
Sligpt	0.00%	8.47%	91.53%	2.20	0.00	0.00%	8.47%	91.53%	1.80	0.00	1.69%
One-Shot	1.69%	15.25%	83.06%	3.00	0.03	5.08%	16.95%	77.97%	1.60	0.10	3.39%
CoT	5.08%	6.78%	88.14%	1.00	0.10	6.78%	10.17%	83.05%	1.00	0.13	1.69%

Table 10. Detailed precision and recall statistics for the overall evaluations on the benchmark dataset. Values in each cell denote Precision/Recall.

	Baseline	Sligpt	One-Shot	CoT
Stage 1: LLM-assisted Base Analyzers				
Mythril	0.80/0.31	0.97/0.40	0.93/0.28	0.95/0.46
Slither	0.51/0.12	0.95/0.29	0.93/0.26	0.95/0.41
SmartCheck	0.70/0.07	0.97/0.29	0.95/0.23	0.97/0.41
Stage 2: Ensemble Strategies				
WI	0.96/0.25	0.97/0.38	0.93/0.27	0.98/0.49
OS	0.85/ 0.40	0.96/ 0.41	0.92/ 0.34	0.95/ 0.52

the proportion of actual vulnerabilities successfully detected within these contracts. Therefore, we provide detailed statistics for them across all evaluations (Tables 10–15).

11 Model Fine-Tuning and Convergence Analysis

For the ZKP-specific semantic alignment model proposed in the main text, we employed LLaMa-3.2-3B [6] as the backbone. It was fine-tuned using Low-Rank Adaptation (LoRA) [10] ($r = 16$, $\alpha = 32$, dropout rate 0.05) for up to 10 epochs (cosine annealing learning rate, peak 1×10^{-4}) with validation-based early stopping to prevent overfitting. The fine-tuning involved a specialized corpus of

Table 11. Detailed precision and recall statistics for the evaluations on the ZKP-based smart contract sub-dataset. Values in each cell denote Precision/Recall.

	Baseline	Sligpt	One-Shot	CoT
<i>Panel A: Configurations with GPT-3.5</i>				
Stage 1: LLM-assisted Base Analyzers				
Mythril	0.36/0.04	0.88/ 0.10	0.94/ 0.13	0.97/0.26
Slither	0.78/ 0.08	0.83/0.09	0.87/0.12	0.97/0.28
SmartCheck	1.00/0.08	1.00/0.08	0.95/0.08	1.00/0.30
Stage 2: Ensemble Strategies				
WI	1.00/0.16	1.00/0.17	1.00/0.17	1.00/0.28
OS	0.84/ 0.17	0.94/ 0.20	0.95/ 0.18	1.00/0.33
<i>Panel B: Configurations with DeepSeek-V3</i>				
Stage 1: LLM-assisted Base Analyzers				
Mythril	0.93/0.12	–	0.59/0.14	0.10/0.05
Slither	0.07/0.02	–	0.79/ 0.32	0.06/0.04
SmartCheck	0.50/0.02	–	0.81/0.29	0.06/0.04
Stage 2: Ensemble Strategies				
WI	1.00/0.16	–	1.00/0.34	1.00/0.23
OS	0.48/ 0.16	–	0.78/ 0.43	0.08/0.06
<i>Panel C: Alternative Ensemble Strategies</i>				
Game-Theoretic	1.00/0.16	1.00/0.17	1.00/0.09	1.00/0.27
Randomization	1.00/0.14	1.00/0.15	1.00/0.14	1.00/0.28

approximately 480 contract pairs and their corresponding hints, following prompt styles consistent with open-source GitHub data [11]. This corpus includes both open-source repositories [11] and a custom-built dataset tailored for ZKP-based contracts, manually created by experts from separate source repositories like zkSync and LayerZero, sourced from a controlled vulnerability pool as described above. To ensure data independence, all fine-tuning samples are excluded from the evaluation benchmark dataset.

Figure 6 presents the training and validation loss throughout fine-tuning. Both losses decrease consistently and remain closely aligned, without a persistent widening validation gap. These results indicate stable convergence and show no evident signs of overfitting.

References

- [1] ChatGPT. 2024. *Pricing Details of ChatGPT API*. Retrieved January 20, 2026 from <https://platform.openai.com/docs/pricing>
- [2] DeepSeek. 2025. *Pricing Details of DeepSeek API*. https://api-docs.deepseek.com/quick_start/pricing
- [3] Thomas Durieux, João F. Ferreira, Rui Abreu, and Pedro Cruz. 2020. Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In *ICSE*. 530–541.
- [4] David Gabay, Kemal Akkaya, and Mumin Cebe. 2020. Privacy-Preserving Authentication Scheme for Connected Electric Vehicles Using Blockchain and Zero Knowledge Proofs. *IEEE Trans. Veh. Technol.* 69, 6 (2020), 5760–5772.
- [5] Asem Ghaleb and Karthik Pattabiraman. 2020. How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection. In *ISSTA*. 415–427.
- [6] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. *arXiv e-prints* (2024), arXiv:2407.21783.
- [7] NCC Group. 2018. *Decentralized Application Security Project*. <https://dasp.co>
- [8] Zaher Haddad. 2024. Enhancing privacy and security in 5G networks with an anonymous handover protocol based on blockchain and zero knowledge proof. *Computer Networks* 250 (2024), 110544.

Table 12. Detailed precision and recall statistics for the evaluations on the SmartBugs Curated sub-dataset. Values in each cell denote Precision/Recall.

	Baseline	Sligpt	One-Shot	CoT
<i>Panel A: Configurations with GPT-3.5</i>				
Stage 1: LLM-assisted Base Analyzers				
Mythril	0.43/ 0.15	0.98 /0.36	0.78/0.21	0.81/ 0.36
Slither	0.10/0.04	0.98 /0.36	1.00 /0.37	0.88/0.34
SmartCheck	0.59 /0.10	0.98 / 0.40	0.98/ 0.41	0.92 /0.35
Stage 2: Ensemble Strategies				
WI	1.00 / 0.22	1.00 /0.39	1.00 /0.35	1.00 /0.35
OS	0.46/0.19	0.98/ 0.42	0.97/ 0.47	0.82/ 0.37
<i>Panel B: Generalization to Other Base Analyzers</i>				
Stage 1: LLM-assisted Base Analyzers				
Honeybadger	0.33/0.01	0.93/0.37	0.95/0.39	0.74/0.34
Manticore	0.80 / 0.15	0.96/0.38	1.00 / 0.48	0.89/0.35
Osiris	0.34/0.09	1.00 /0.38	1.00 /0.36	0.86/ 0.36
Oyente	0.46/ 0.15	0.98/ 0.41	1.00 /0.44	0.90 /0.33
Stage 2: Ensemble Strategies				
WI	–	1.00 /0.37	1.00 /0.48	1.00 /0.36
OS	–	1.00 / 0.48	1.00 / 0.59	0.81/ 0.38

Table 13. Detailed precision and recall statistics for the evaluations on the SolidiFI Benchmark sub-dataset. Values in each cell denote Precision/Recall.

	Baseline	Sligpt	One-Shot	CoT
<i>Panel A: Configurations with GPT-3.5</i>				
Stage 1: LLM-assisted Base Analyzers				
Mythril	0.95/ 0.77	1.00 / 0.85	0.99/ 0.57	1.00 / 0.91
Slither	1.00 /0.27	1.00 /0.59	1.00 /0.41	1.00 /0.80
SmartCheck	0.79/0.07	1.00 /0.57	1.00 /0.26	1.00 /0.82
Stage 2: Ensemble Strategies				
WI	1.00 /0.36	1.00 /0.66	1.00 /0.37	1.00 /0.85
OS	0.98/ 0.71	1.00 / 0.69	1.00 / 0.47	1.00 / 0.88
<i>Panel B: Generalization to Other Base Analyzers</i>				
Stage 1: LLM-assisted Base Analyzers				
Manticore	0.90/ 0.31	1.00 /0.59	1.00 /0.68	1.00 /0.81
Securify	0.90/0.25	1.00 / 0.71	1.00 /0.36	1.00 /0.79
Oyente	0.92 /0.12	1.00 /0.62	1.00 / 0.98	1.00 / 0.99
Stage 2: Ensemble Strategies				
WI	–	1.00 /0.68	1.00 /0.60	1.00 /0.86
OS	–	1.00 / 0.73	1.00 / 0.68	1.00 / 0.88

- [9] Jiaqi He, Revan MacQueen, Natalie Bombardieri, et al. 2023. Finding an Optimal Set of Static Analyzers To Detect Software Vulnerabilities. In *ICSME*. 463–473.
- [10] Edward J Hu, Yelong Shen, Phillip Wallis, et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR* 1, 2 (2022), 3.

Table 14. Detailed precision and recall statistics for the evaluations on the DAppSCAN sub-dataset. Values in each cell denote Precision/Recall.

	Baseline	Sligpt	One-Shot	CoT
Stage 1: LLM-assisted Base Analyzers				
Mythril	0.00/0.00	0.00/0.00	0.00/0.00	0.33/0.02
Slither	0.00/0.00	0.00/0.00	0.14/0.02	0.50 /0.05
SmartCheck	0.00/0.00	0.00/0.00	0.40 /0.04	0.25/0.02
Stage 2: Ensemble Strategies				
WI	0.00/0.00	0.00/0.00	0.10/0.02	0.43/0.05
OS	0.00/0.00	0.00/0.00	0.23/ 0.06	0.40/ 0.08

Table 15. Detailed precision and recall statistics for the evaluation of various state-of-the-art static analysis approaches for detecting vulnerabilities in smart contracts across each sub-dataset. Values in each cell denote Precision/Recall.

Approach	ZKP	SmartBugs	SolidiFI	DAppSCAN
GPT-3.5	0.95/0.09	0.42/0.13	0.43/0.14	0.25/0.02
GPT-4o	0.56/0.05	0.27/0.11	0.75/0.05	0.00/0.00
Smartbugs	0.29/0.09	0.13/0.07	0.14/0.22	0.00/0.00
Sligpt	0.83/0.09	0.98 /0.36	1.00 /0.59	0.00/0.00
GPTScan	0.77/0.14	0.22/0.07	0.51/0.33	0.20/0.02
ELSA	1.00 / 0.33	0.97/ 0.47	1.00 / 0.88	0.40 / 0.08

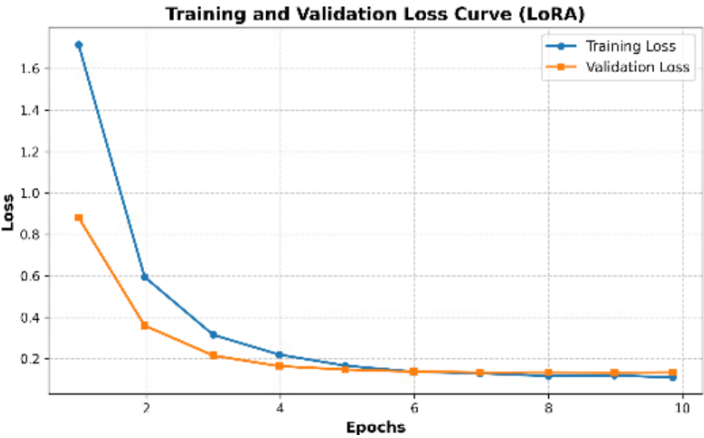


Fig. 6. Training and validation loss curves for LLaMa-3.2-3B over 10 epochs of LoRA fine-tuning.

[11] Xinjian Ji and Bingqian Wang. 2024. *Chinese Large Model Training Dataset for Smart Contract Vulnerability Repair*. Retrieved January 20, 2026 from <https://github.com/wizonce9/Chinese-LLM-Training-Dataset-for-Smart-Contract-Vulnerability-Repair>

[12] Rakesh Kumar and Amrita Chaturvedi. 2024. Software Bug Prediction Using Reward-Based Weighted Majority Voting Ensemble Technique. *IEEE Transactions on Reliability* 73, 1 (2024), 726–740.

[13] Xiaoqiang Sun, F. Richard Yu, Peng Zhang, Zhiwei Sun, Weixin Xie, and Xiang Peng. 2021. A Survey on Zero-Knowledge Proof in Blockchain. *IEEE Network* 35, 4 (2021), 198–205.

[14] Gerhard Wagner. 2022. *Smart Contract Weakness Classification (SWC)*. Retrieved January 20, 2026 from <http://swcregistry.io>

- [15] Hongbo Wen, Jon Stephens, Yanju Chen, Kostas Ferles, et al. 2024. Practical Security Analysis of Zero-Knowledge Proof Circuits. In *USENIX Security* 24. 1471–1487.
- [16] Dinghuai Zhang, Hongyang Zhang, Aaron Courville, et al. 2022. Building Robust Ensembles via Margin Boosting. In *ICML*, Vol. 162. 26669–26692.
- [17] Zibin Zheng, Jianzhong Su, Jiachi Chen, et al. 2024. DAppSCAN: Building Large-Scale Datasets for Smart Contract Weaknesses in DApp Projects. 50, 6 (June 2024), 1360–1373.
- [18] Lu Zhou, Abebe Diro, Akanksha Saini, Shahriar Kaisar, and Pham Cong Hiep. 2024. Leveraging zero knowledge proofs for blockchain-based identity sharing: A survey of advancements, challenges and opportunities. *Journal of Information Security and Applications* 80 (2024), 103678.

Received 2026-01-30; accepted 2026-04-16